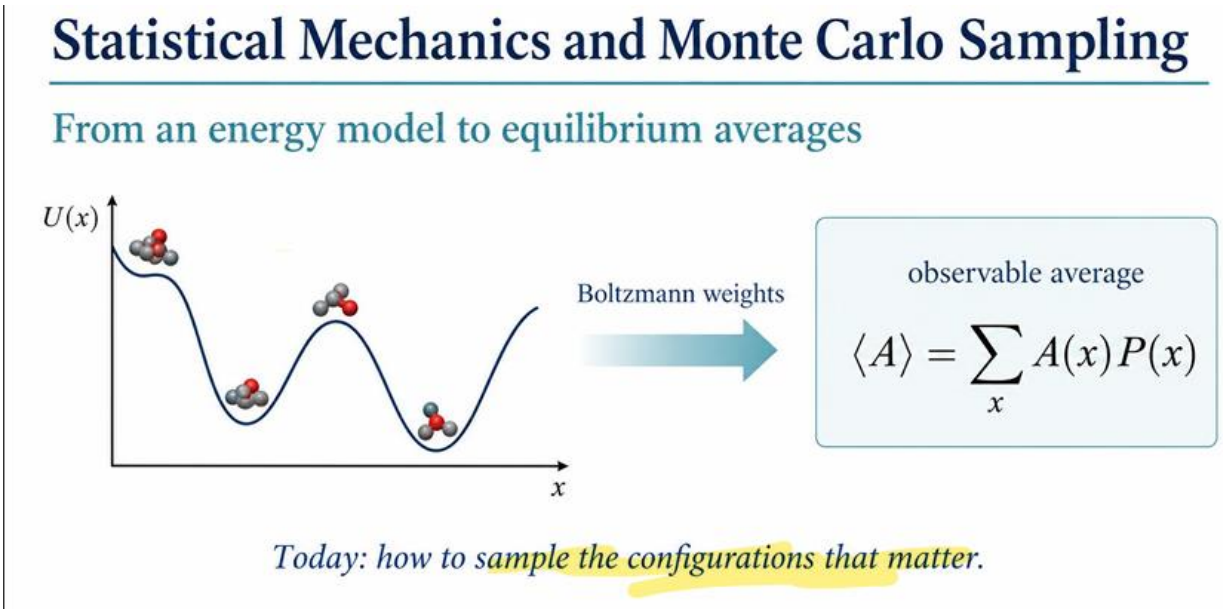
Week 2 –

Slide 1:



This slide illustrates the core bridge between physical energy models and macroscopic properties in statistical mechanics, highlighting why standard uniform sampling fails and Monte Carlo sampling is necessary.

1. The Potential Energy Landscape $U(x)$

- **Horizontal Axis (x):** Represents the **configuration space** (e.g., atomic coordinates, molecular conformers, or orientations).

- **Vertical Axis ($U(x)$):** Represents the **potential energy** of a given configuration x.

- **Energy Wells vs. Peaks:**

- Low points (valleys/wells) correspond to **stable, low-energy configurations**.

- High points (barriers/peaks) correspond to **unfavorable, high-energy states**.

2. Boltzmann Distribution & Boltzmann Weights

In thermodynamic equilibrium at temperature T, the probability $P(x)$ of observing a specific configuration x is given by the **Boltzmann distribution**:

$$P(x) = \frac{e^{-\beta U(x)}}{Z}$$

Where:

- $\beta = \frac{1}{k_B T}$ (k_B is the Boltzmann constant, T is temperature).

- $e^{-\beta U(x)}$ is the **Boltzmann weight**.

- $Z = \sum_x e^{-\beta U(x)}$ is the **partition function** (the normalizing factor).

Key physical intuition: Low-energy states ($U(x) \ll 0$) have exponentially higher probability weights, while high-energy states ($U(x) \gg 0$) have negligible weights.

3. Calculating Observable Averages $\langle A \rangle$

To predict a measurable physical property $\langle A \rangle$ (such as average energy, pressure, or binding affinity), we compute an ensemble average:

$$\langle A \rangle = \sum_{x} A\,(x)\,P(x)$$

- $A(x)$ is the value of the physical quantity in configuration x.

- $P(x)$ weighs each state by its thermodynamic likelihood.

4. The Problem & "Configurations That Matter"

- **Dimensionality Challenge:** For realistic physical systems (e.g., proteins or fluids), the total number of possible configurations x is astronomically vast ($10^{100}+$ states).

- **Efficiency Bottleneck:** Most random configurations have extremely high energies (e.g., overlapping atoms), leading to $e^{-\beta U(x)} \approx 0$. Sampling configurations uniformly would waste almost all computational time evaluating states with zero statistical weight.

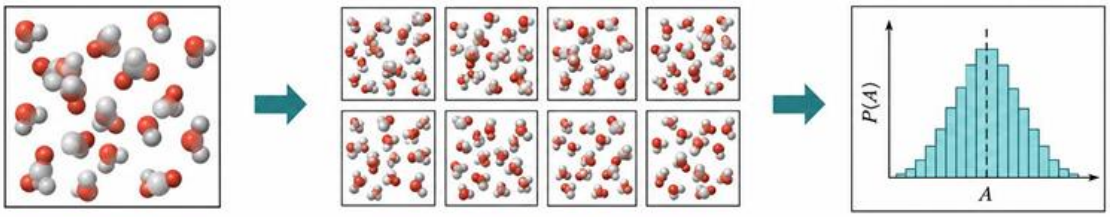
- **Monte Carlo Solution:** Instead of uniform random sampling, algorithms like **Metropolis-Hastings Monte Carlo** generate configurations directly according to their Boltzmann weight $P(x)$. This ensures computational effort is concentrated strictly on the low-energy valleys—the *"configurations that matter"*.

Slide 2:

WEEK 2

From one configuration to a thermodynamic property

A single configuration is not the property.



one microstate x | many accessible configurations | estimate $\langle A \rangle$

sample ensemble snapshot

- Properties are statistical.
- Configurations have different probabilities.
- Sampling replaces impossible enumeration.

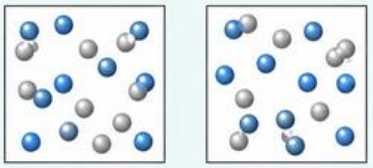
We need rules for which configurations count, and how much.

Slide 3:

WEEK 2

Microstates, macrostates, and observables

Microstates



complete microscopic specification

many microstates can share the same macrostate

Macrostate

N, V, T

$\langle E \rangle$

density ρ

composition

Observable: $A(\mathbf{x})$

$\langle A \rangle$ = average over sampled microstates

Statistical mechanics connects microscopic states to measurable macroscopic behavior.

The fundamental link between **microstates** and **macrostates**, which serves as the core foundation of statistical mechanics.

1. Microstates (The Microscopic View)

- **Definition:** A microstate provides a complete microscopic specification of a system, defining the exact position and momentum of every single particle at a given instant.

- **Illustration:** The four boxes on the left display four distinct microscopic configurations, where the blue and gray atoms occupy different spatial positions while representing the same physical system.

2. Macrostates (The Thermodynamic View)

- **Definition:** A macrostate describes the overall condition of the system from a macroscopic perspective using thermodynamic variables.

- **Key Properties:** It is defined by parameters such as particle number (N), volume (V), temperature (T), average energy ($\langle E \rangle$), density (ρ), and system composition.

3. The Bridge: From Micro to Macro

- **Many-to-One Mapping:** A vast number of unique microstates can share the exact same macrostate, as indicated by the central arrow. For example, atoms can continuously shift positions inside a box without changing the system's overall temperature or volume.

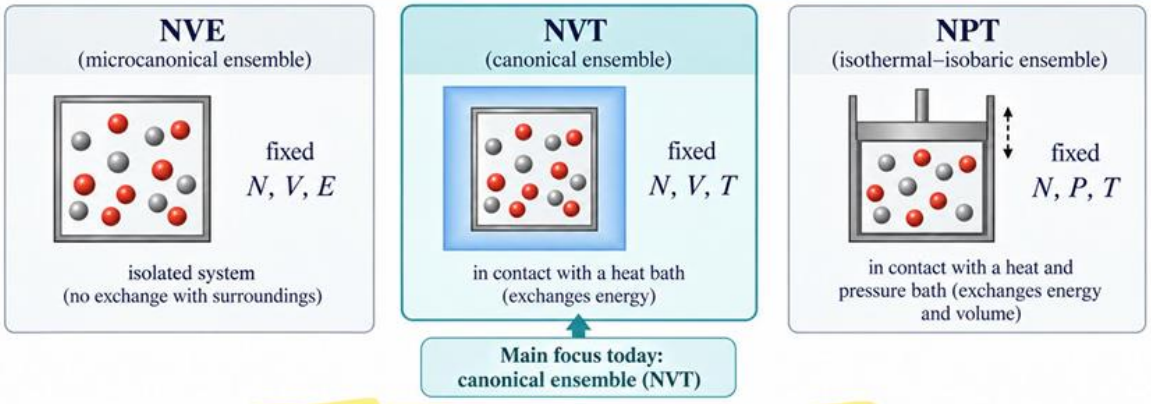
- **Observables:** Any measurable macroscopic property $\langle A \rangle$ is calculated mathematically as an ensemble average over the sampled microstates.

Core Takeaway: *"Statistical mechanics connects microscopic states to measurable macroscopic behavior."* In computer modeling, this means we can predict real-world bulk properties of materials by simulating and averaging atomic-level configurations.

Slide 4:

WEEK 2

What is an ensemble?



- Ensemble = many microscopic realizations under the same macroscopic conditions.
- Different ensembles correspond to different constraints.
- Today we will sample the canonical ensemble.

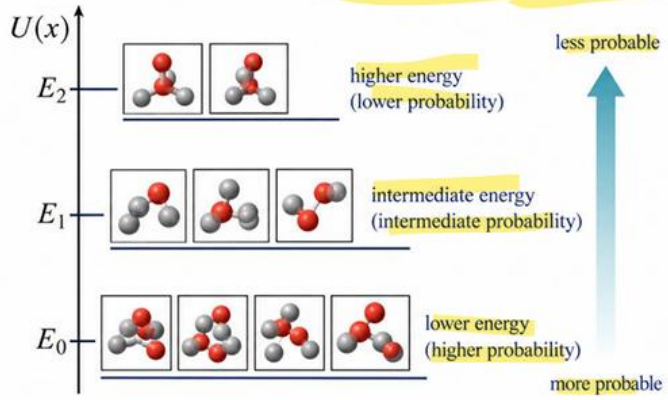
An ensemble provides the statistical framework behind thermodynamic averages.

Slide 5:

WEEK 2

The canonical ensemble and the Boltzmann distribution

Not all microstates are equally probable in the canonical ensemble.



Boltzmann distribution

$$P(x) = \frac{\exp[-\beta U(x)]}{Z}$$

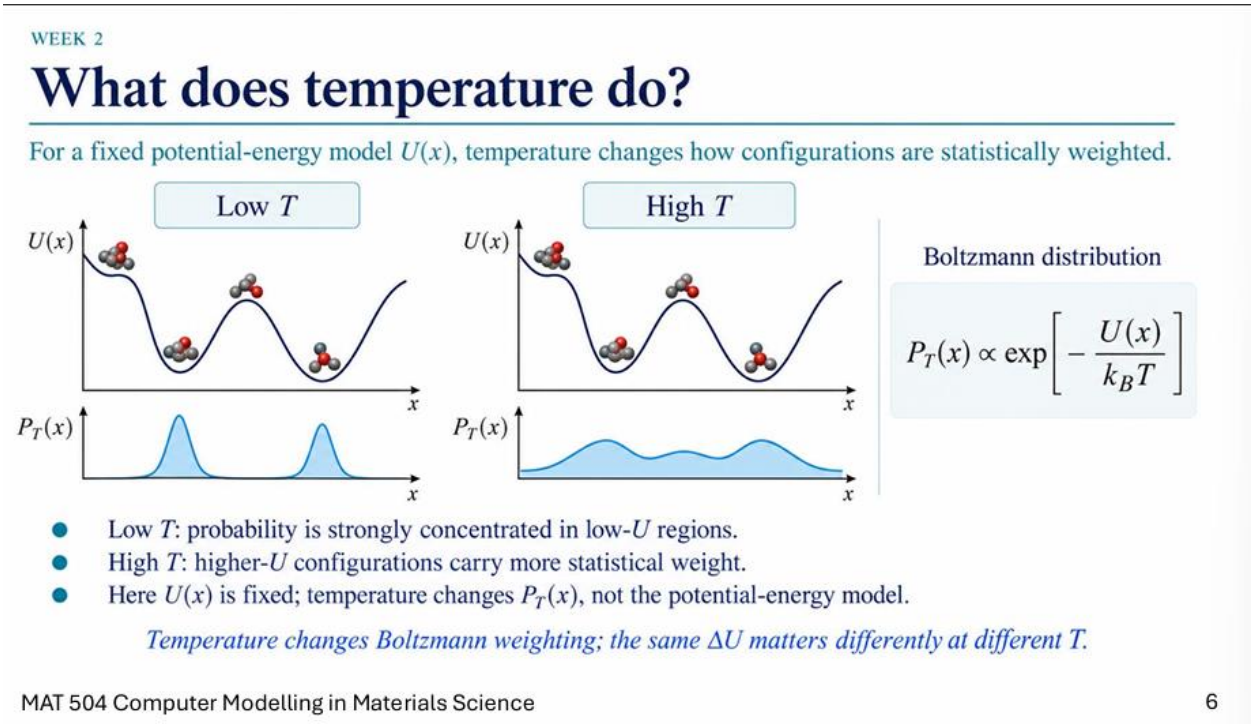
$$\beta = \frac{1}{k_B T}$$

Z = partition function (normalization)

- Probability depends on energy and temperature.
- Lower-energy configurations carry larger statistical weight.
- At finite temperature, high-energy states are suppressed, not forbidden.

Equilibrium properties depend on weighted averages, not just the minimum-energy state.

Slide 6:



What Does Temperature Do?

This slide explains how temperature governs the statistical probability of finding a physical system in various molecular configurations for a given energy landscape.

1. The Core Principle

For a fixed potential energy surface $U(x)$, changing the temperature T alters how different configurations x are **statistically weighted** without modifying the underlying energy landscape itself.

2. The Boltzmann Distribution

The probability $P_T(x)$ of observing a specific configuration x at temperature T is proportional to its Boltzmann weight:

$$P_T(x) \propto \exp\left[-\frac{U(x)}{k_B T}\right]$$

- $U(x)$: Potential energy of configuration x.

- k_B: Boltzmann constant.

- T: Absolute temperature in Kelvin.

3. Low T vs. High T Behavior

- **Low Temperature (Low T):**

    - k_BT is small, making the exponent $-\frac{U(x)}{k_BT}$ heavily penalty-heavy for high energies.

    - Probability $P_T(x)$ is **sharply concentrated** only in the lowest-energy valleys ($U(x) \approx$ minimum).

    - *Physical meaning:* At low temperatures, molecules freeze into low-energy ground states and rarely overcome energy barriers.

- **High Temperature (High T):**

    - k_BT is large, flattening the exponent across different energies.

    - Higher-energy configurations carry **significantly more statistical weight**, spreading out the probability distribution $P_T(x)$.

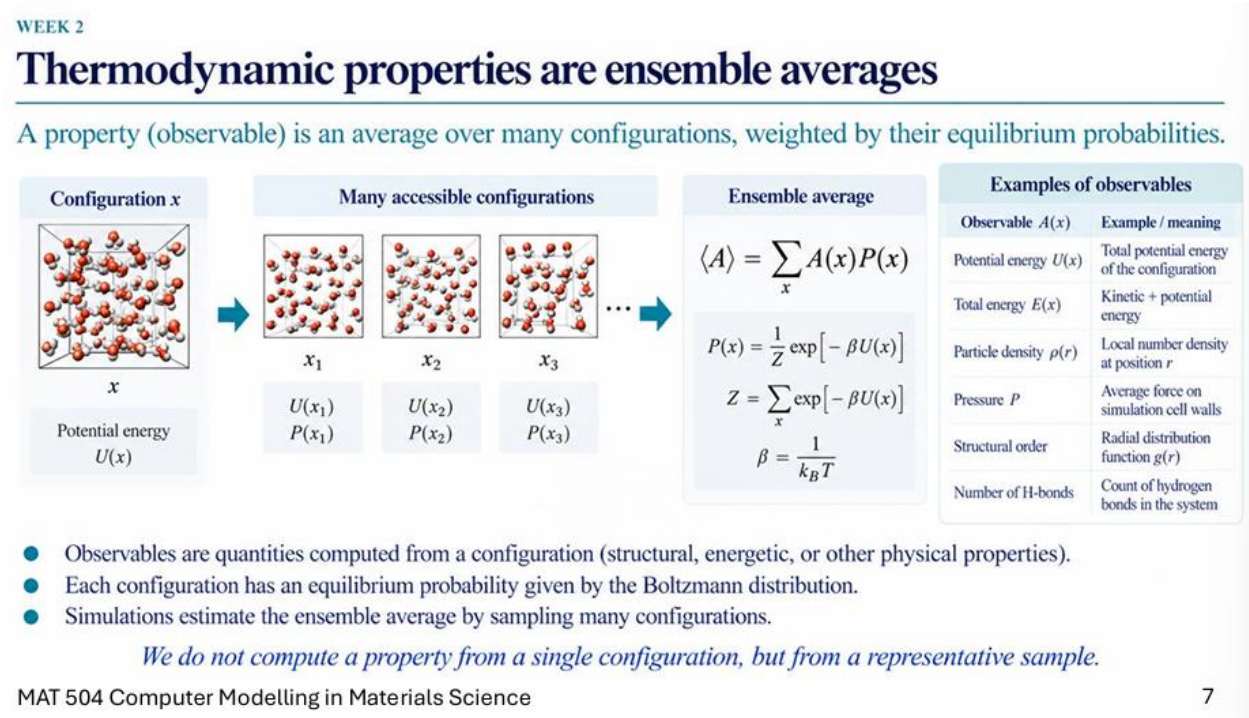
- *Physical meaning:* Thermal energy enables the system to cross energy barriers and explore higher-energy states (e.g., melting, unfolding, or phase transitions).

Key Takeaway

"Temperature changes Boltzmann weighting; the same ΔU matters differently at different T."

The potential energy model $U(x)$ remains identical, but temperature dictates whether energy barriers act as strict walls (at low T) or minor bumps (at high T).

Slide 7:



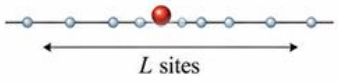
Slide 8:

WEEK 2

Why can't we enumerate configuration space?

The number of possible configurations grows explosively with the number of degrees of freedom.

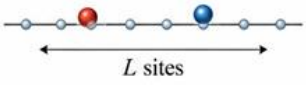
One particle on a 1D grid



$\Omega = L$

(L possible positions)

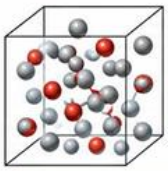
Two particles on a 1D grid



$\Omega = L^2$

(each particle can be at any of L sites)

Many-particle molecular system



N atoms

$3N$ coordinates

(x, y, z for each atom)

Configuration space dimension = $3N$

$\Omega \sim L^{3N}$

(grows exponentially with N)

Exhaustive enumartion is impossible, equilibrium properties must be estimated by sampling.

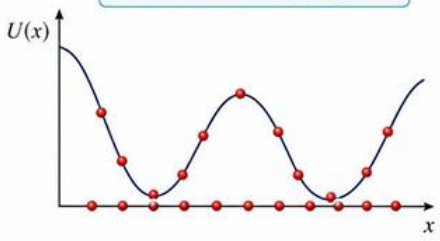
Slide 9:

WEEK 2

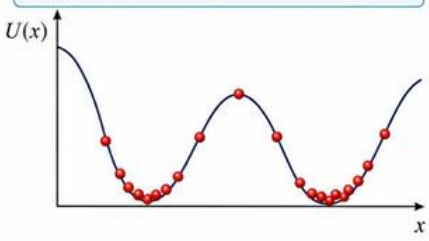
Importance sampling: spend effort where probability is high

We do not need every configuration equally; we need configurations with the correct statistical frequency.

Uniform sampling



Importance sampling (Boltzmann weighted)



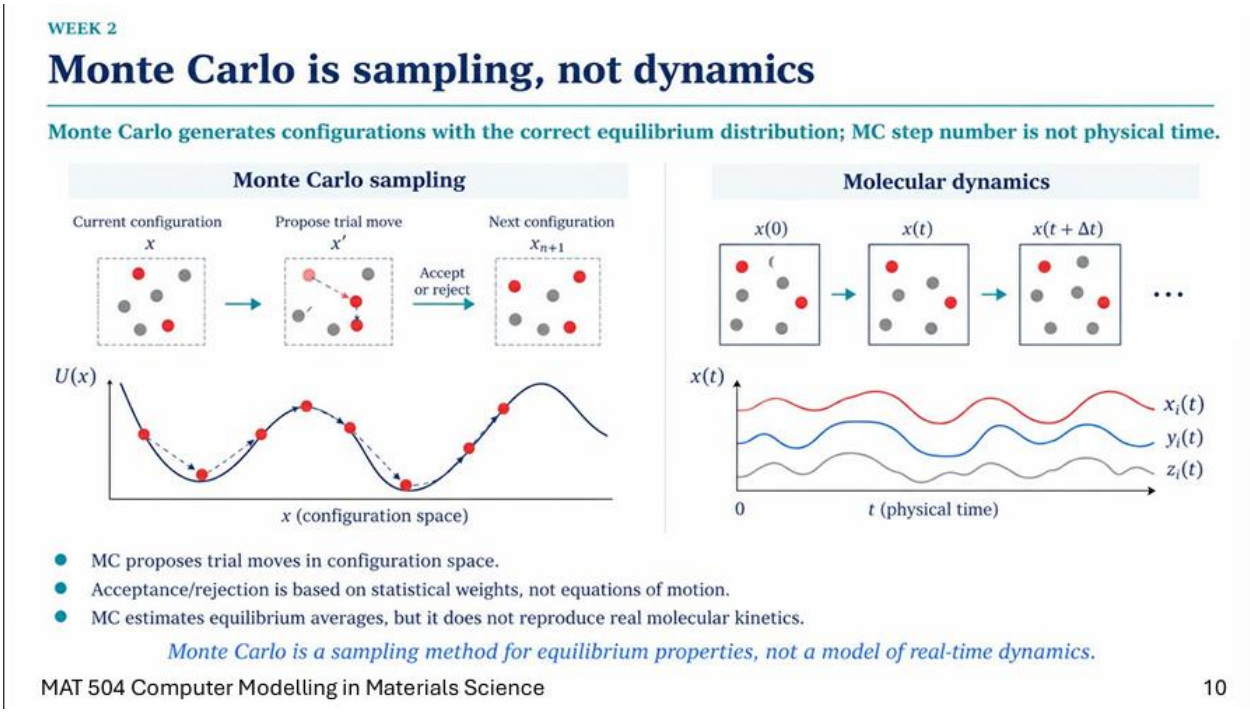
Boltzmann distribution

$$P(x) \propto \exp\left[-\frac{U(x)}{k_B T}\right]$$

- Uniform sampling wastes effort in low-probability regions.
- Importance sampling concentrates effort where equilibrium probability is large.
- Correct averages are preserved because samples appear with the target distribution.

Importance sampling reduces wasted effort while preserving the correct equilibrium distribution.

Slide 10:



1. Key Concept: Sampling vs. Dynamics

"Monte Carlo is a sampling method for equilibrium properties, not a model of real-time dynamics."

- **Monte Carlo (MC) Sampling:** Generates configurations according to their thermodynamic probability (the Boltzmann distribution). The step index n is simply a counter, **not physical time**.

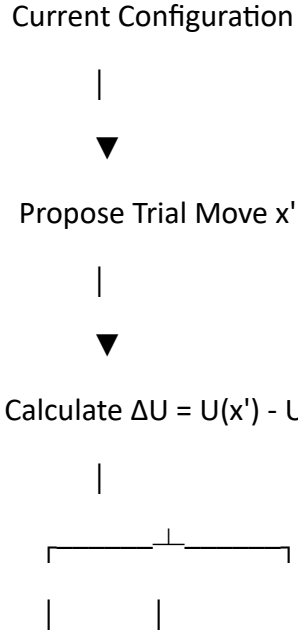
- **Molecular Dynamics (MD):** Solves Newton's equations of motion ($F = ma$) to trace the continuous, deterministic trajectory of atoms over actual **physical time (t)**.

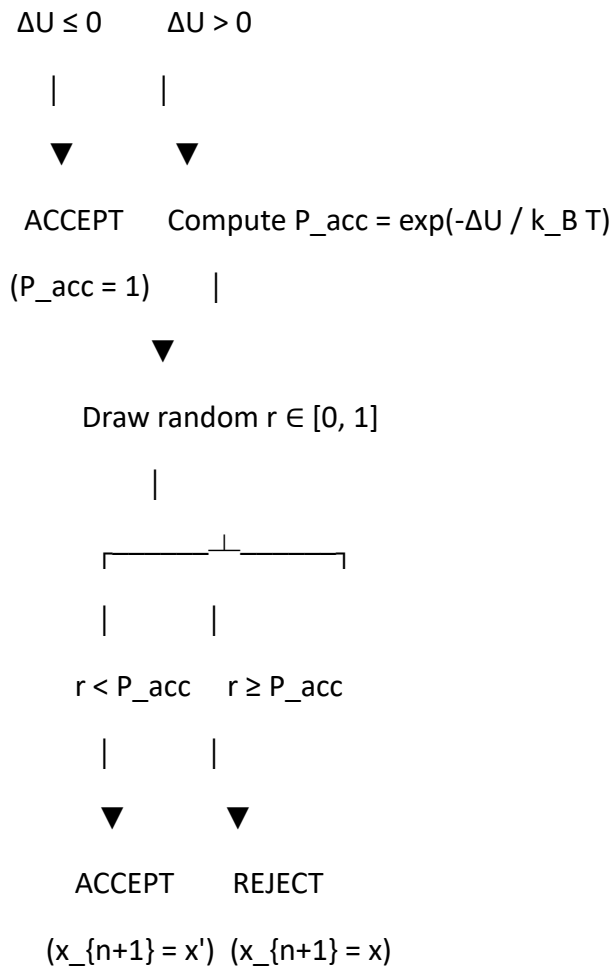
2. Comparison Summary

Feature	Monte Carlo (MC)	Molecular Dynamics (MD)
Motion Basis	Stochastic trial moves ($x \to x'$)	Physical equations of motion ($F = ma$)
Time Axis	Step count n (no physical time)	Continuous physical time t
Decision Rule	Statistical acceptance/rejection	Numerical integration of trajectories
Primary Goal	Equilibrium thermodynamic averages	Time-dependent kinetics & dynamics

3. How the Accept / Reject Mechanism Works

In the standard **Metropolis Monte Carlo** algorithm, trial moves are accepted or rejected based on potential energy changes to ensure the system visits configurations with correct Boltzmann weights.





Step-by-Step Breakdown

1. **Propose a Trial Move ($x \to x'$):** A random displacement is applied to one or more atoms to propose a new configuration x'.

2. **Calculate the Energy Change (ΔU):**

$$\Delta U = U(x') - U(x)$$

3. **Apply the Metropolis Acceptance Criterion:**

   - **Case 1: Downhill Move ($\Delta U \leq 0$)** If the proposed state has equal or lower potential energy, the move is **always accepted** ($P_{\text{acc}} = 1$).

- **Case 2: Uphill Move ($\Delta U > 0$)**

If the proposed state has higher potential energy, it is **accepted conditionally** with probability:

$$P_{\text{acc}} = \exp\left(-\frac{\Delta U}{k_B T}\right)$$

4. **Stochastic Decision:**

- A uniform random number $r \in [0,1]$ is drawn.

- If $r < P_{\text{acc}}$, the move is **accepted** ($x_{n+1} = x'$).

- If $r \geq P_{\text{acc}}$, the move is **rejected** ($x_{n+1} = x$), and the system retains its previous state for the next step.

Why Accept Higher-Energy Moves?

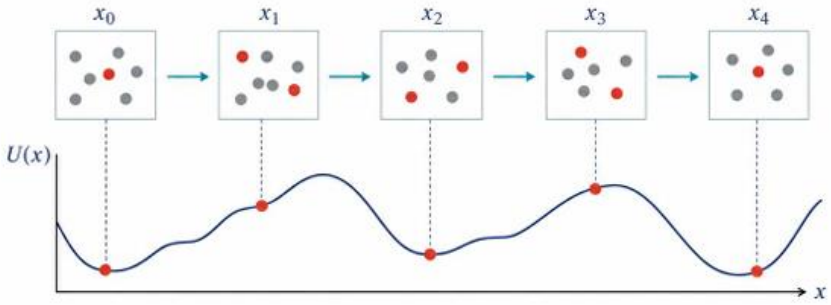
If higher-energy moves were always rejected, the simulation would get trapped in the nearest local minimum. Allowing occasional uphill moves proportional to $\exp\left(-\frac{\Delta U}{k_B T}\right)$ enables the system to cross energy barriers and explore the full configuration space according to correct thermal fluctuations.

Slide 11:

WEEK 2

Markov chains: generating configurations sequentially

Each new configuration is generated from the current one; the long-run distribution is designed to be the target ensemble.



Markov property

$$P(x_{n+1} \mid x_n, x_{n-1}, \ldots, x_0) = P(x_{n+1} \mid x_n)$$

Target stationary distribution

$$P(x) \propto \exp\left[-\beta U(x)\right]$$

- The next step depends on the current state, not the full history.
- Transition rules define a Markov chain in configuration space.
- We choose the rules so the stationary distribution is the Boltzmann distribution.

A Monte Carlo simulation constructs a chain whose long run behavior reproduces the target ensemble.

1. What is the Markov Property? (No Memory)

In a **Markov Chain**, the probability of moving to the next state (x_{n+1}) depends **only on the current state** (x_n), not on how the system got there (the history $x_0, x_1, \ldots, x_{n-1}$):

$$P(x_{n+1} \mid x_n, x_{n-1}, \ldots, x_0) = P(x_{n+1} \mid x_n)$$

2. How Markov Chains & Monte Carlo Link Together

1. **The Problem:** We want to sample configurations from the **Boltzmann distribution** $P(x) \propto \exp[-\beta U(x)]$, but direct random sampling is impossible for large systems.

2. **The Markov Chain Engine:** Instead of sampling randomly from scratch, Monte Carlo generates a sequence of configurations ($x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3$ …) where each move is proposed from the current state.

3. **The Target Equilibrium:** We design the transition rules (the Metropolis accept/reject criterion) so that the chain eventually settles into a **stationary distribution** equal to the Boltzmann distribution.

Key Takeaway

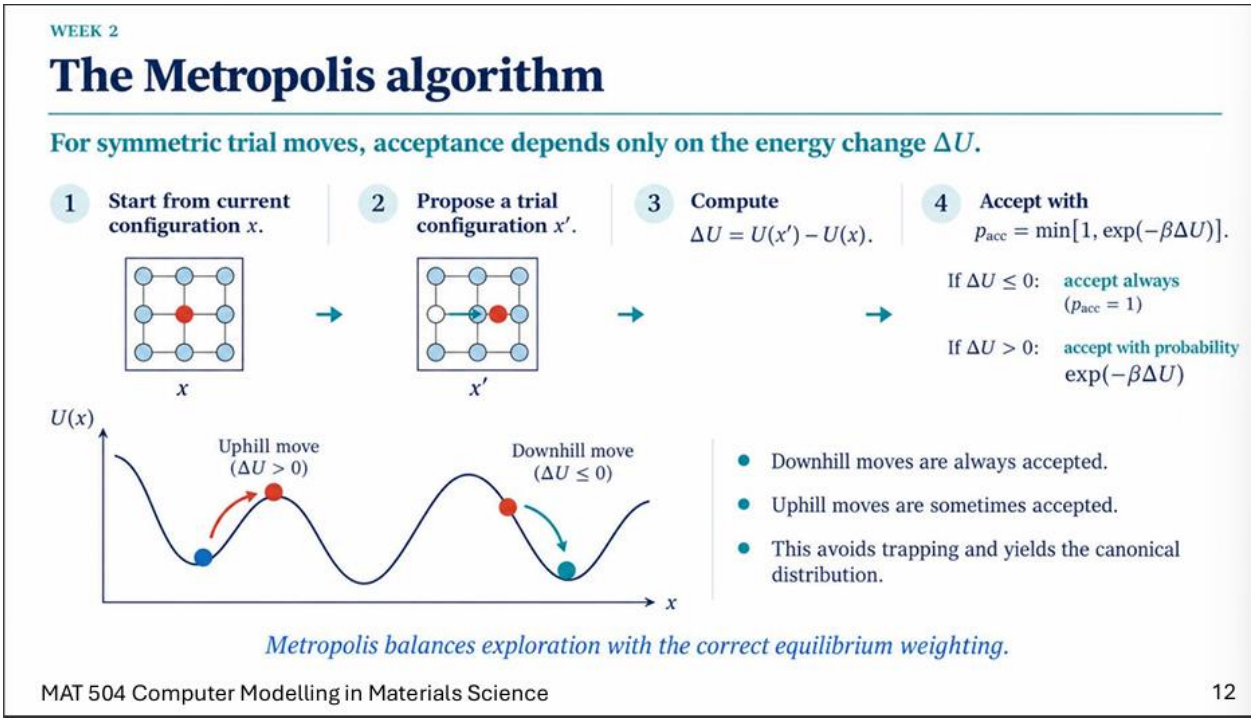
Monte Carlo is the algorithm, and the Markov Chain is the path it takes.

By making small, memoryless step-by-step moves, Monte Carlo builds a Markov chain whose long-term trajectory naturally visits configurations with the correct equilibrium weights.

Class:

Markov property says the next movement just depends on the previous or current step.

Slide 12:



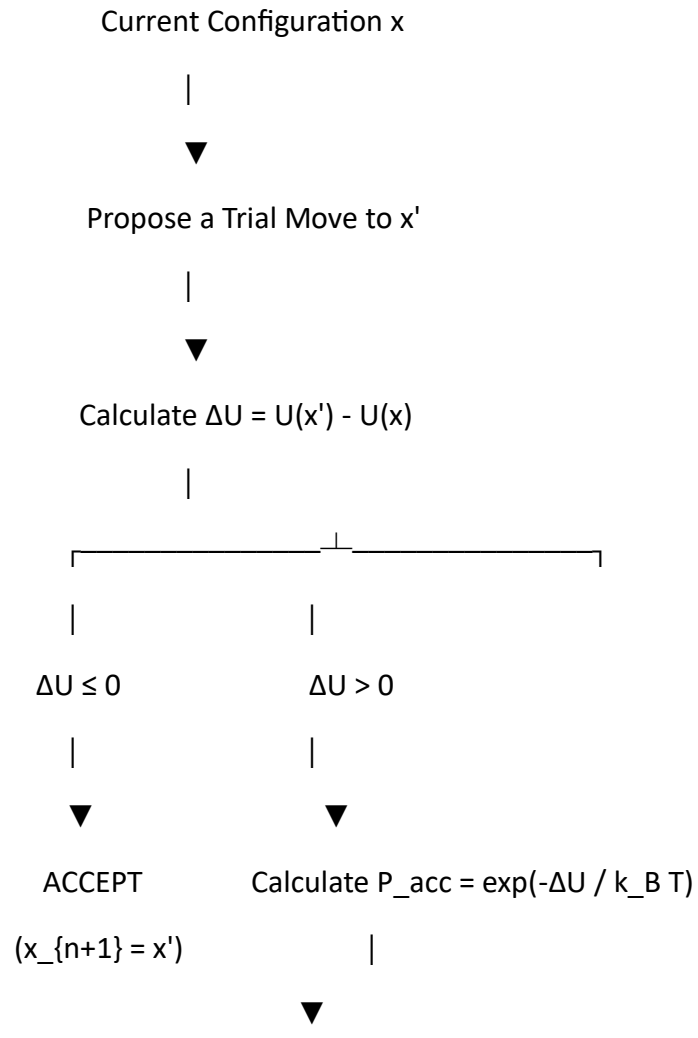
What is the Metropolis Algorithm?

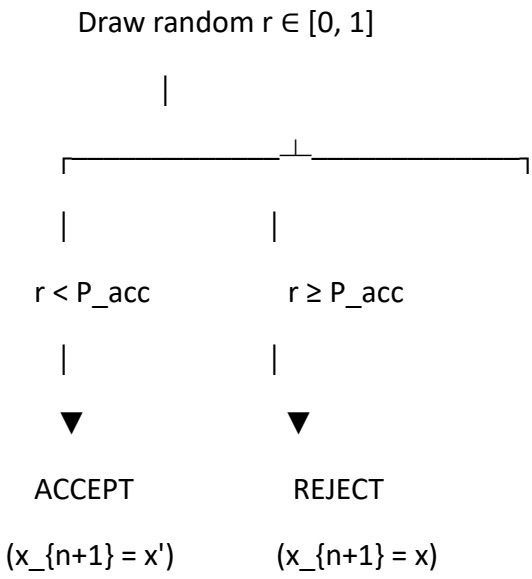
The **Metropolis Algorithm** (often extended as the *Metropolis-Hastings algorithm*) is the core algorithm used in **Monte Carlo simulations** for statistical mechanics and materials science.

It provides a mathematically rigorous way to generate a sequence of configurations (a Markov chain) that samples states according to their thermodynamic probability—the **Boltzmann distribution**.

How It Works (Step-by-Step)

The algorithm operates iteratively starting from a given system configuration x:





1. **Start at Current State (x):** The system begins in a valid state with potential energy $U(x)$.

2. **Propose a Trial Move (x'):** Apply a random modification (e.g., displacing an atom or rotating a molecule) to obtain a candidate state x'.

3. **Compute Energy Difference (ΔU):**

$$\Delta U = U(x') - U(x)$$

4. **Apply the Acceptance Rule:**

   - **Downhill / Equal Energy ($\Delta U \leq 0$):** Always **ACCEPT** the move ($P_{\text{acc}} = 1$).

   - **Uphill Energy ($\Delta U > 0$):** Accept **conditionally** with probability:

$$P_{\text{acc}} = \exp\left(-\frac{\Delta U}{k_B T}\right)$$

5. **Draw Random Number (r):** Pick a uniform random value $r \in [0,1]$:

- If $r < P_{\text{acc}} \rightarrow$ **Accept:** Set $x_{n+1} = x'$.

- If $r \geq P_{\text{acc}} \rightarrow$ **Reject:** Keep $x_{n+1} = x$ (the system stays at x for the next count).

Why Is It Useful?

- **Focuses on "Configurations That Matter":** Instead of evaluating useless high-energy states at random, it efficiently samples states proportional to their Boltzmann weights $P(x) \propto \exp(-\beta U(x)))$.

- **Escapes Local Minima:** By allowing occasional higher-energy moves (scaled by temperature T), the simulation avoids getting stuck in local energy traps.

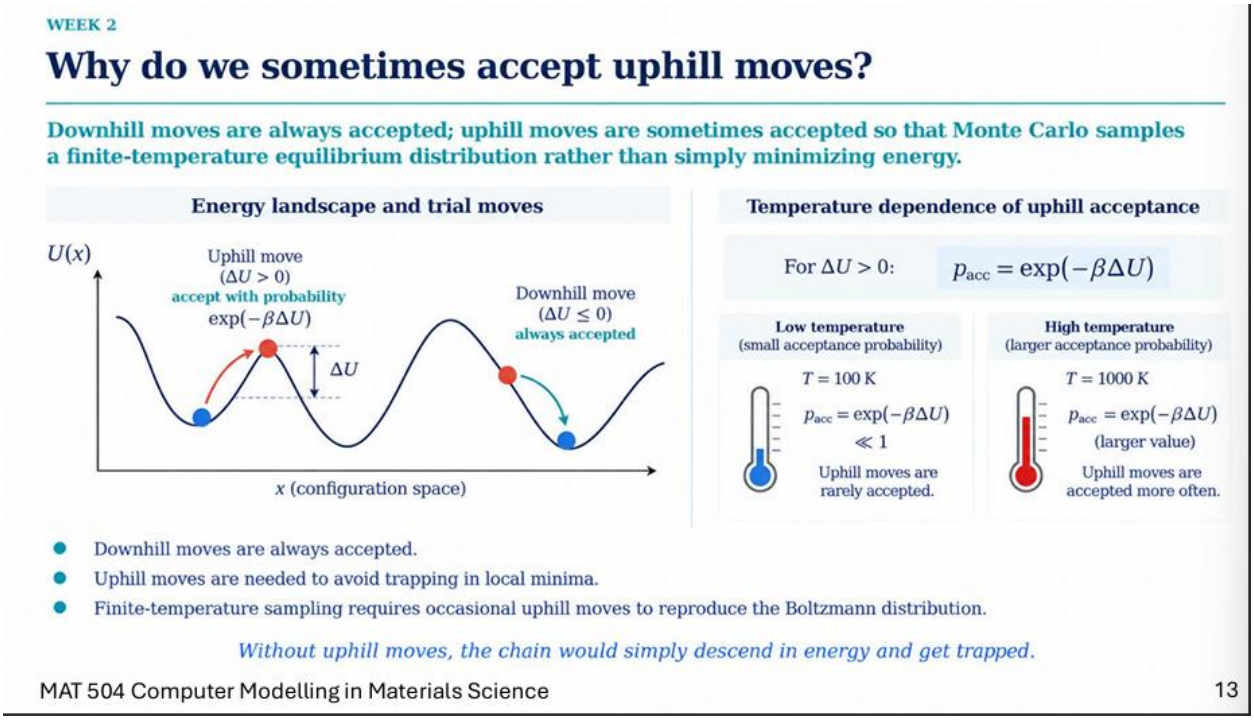
- **Guarantees Convergence:** It satisfies the principle of **detailed balance**, ensuring that the long-term distribution of the generated Markov chain strictly matches the true thermodynamic equilibrium ensemble.

Compare MC (Metropolis) with energy minimization:

AU positive will be rejected for sure; no chance.

The plot is simplified; the real one will be a 3D plot.

Slide 13:



Why Do We Sometimes Accept Uphill Moves?

Why should an algorithm accept a move that increases potential energy?

1. Core Concept

- **Downhill moves ($\Delta U \leq 0$):** Always accepted ($P_{\text{acc}} = 1$).

- **Uphill moves ($\Delta U > 0$):** Accepted conditionally with probability $P_{\text{acc}} = \exp(-\beta \Delta U)$.

The Big Picture: Downhill moves drive the system toward lower energy, but accepting uphill moves ensures Monte Carlo samples a **finite-temperature equilibrium distribution** rather than simply performing energy minimization (steepest descent).

2. Temperature Dependence of Uphill Acceptance

The probability of accepting an uphill move ($\Delta U > 0$) is defined by:

$$P_{\text{acc}} = \exp(-\beta \Delta U) = \exp\left(-\frac{\Delta U}{k_B T}\right)$$

- **Low Temperature ($T = 100$ K):**
    - Thermal energy ($k_B T$) is small.
    - $P_{\text{acc}} \ll 1$ (very small acceptance probability).
    - Uphill moves are **rarely accepted**. The system spends most of its time trapped in low-energy basins.
- **High Temperature ($T = 1000$ K):**
    - Thermal energy ($k_B T$) is large.
    - P_{acc} is significantly larger.
    - Uphill moves are **accepted more often**. Thermal fluctuations allow the system to easily jump over energy barriers and explore higher-energy configurations.

3. Key Takeaways

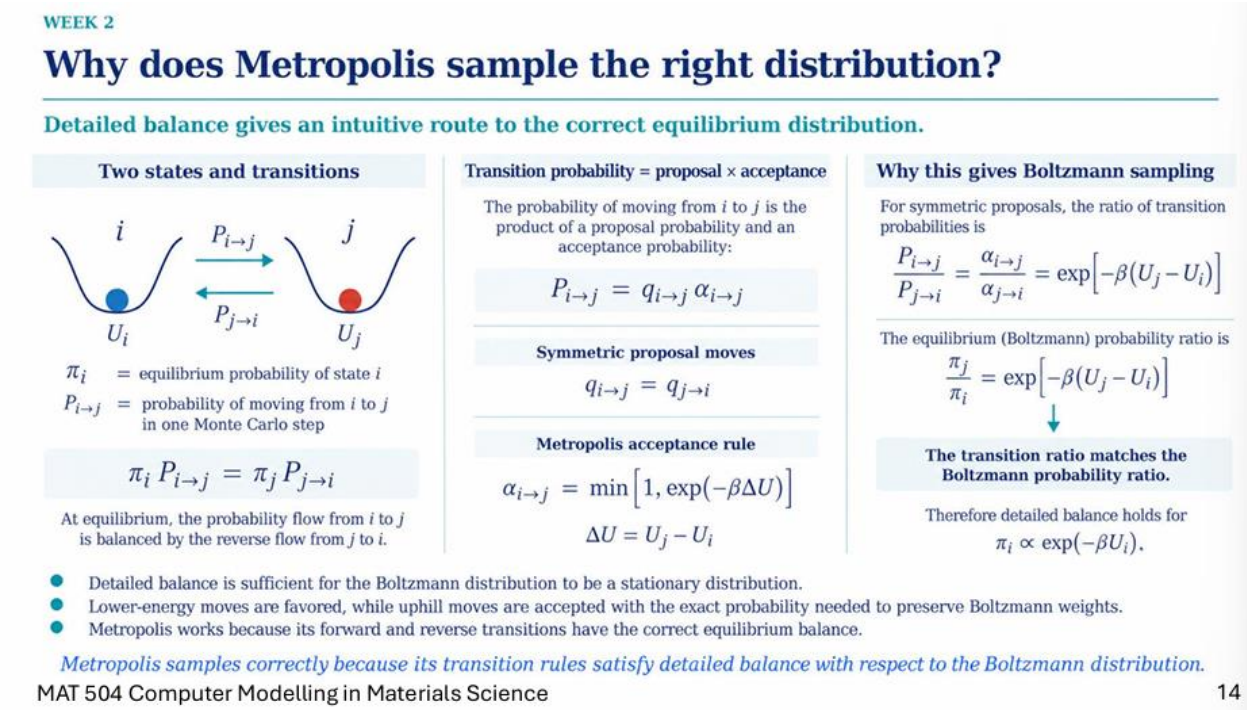
1. **Avoids Local Minima Traps:** Without uphill moves, the simulation would only descend in energy and get permanently stuck in the nearest local minimum.

2. **Reproduces Thermodynamics:** Finite-temperature equilibrium requires that higher-energy states exist according to their Boltzmann weight. Uphill moves allow the system to sample these states with the correct statistical frequency.

Class:

Higher temp = higher prob. At a fixed U.

Slide 14:



1. The Core Concept: Detailed Balance

To ensure that a Monte Carlo simulation eventually samples the correct equilibrium distribution, it must satisfy a principle called **detailed balance**.

- **Definition:** At equilibrium, the probability flux of transitioning from state i to state j must exactly equal the reverse flux from state j to state i:

$$\pi_i P_{i\to j} = \pi_j P_{j\to i}$$

- **Where:**

  - π_i and π_j are the equilibrium probabilities of being in states i and j.

  - $P_{i\to j}$ is the total transition probability of moving from i to j in one Monte Carlo step.

2. Transition Probability Breakdown

The total probability of transitioning from state i to state j is the product of two separate steps:

$$P_{i\to j} = q_{i\to j}\,\alpha_{i\to j}$$

1. **Proposal probability ($q_{i\to j}$):** The probability that the algorithm randomly suggests a move from i to j. (For standard symmetric moves, $q_{i\to j} = q_{j\to i}$).

2. **Acceptance probability ($\alpha_{i\to j}$):** The Metropolis acceptance rule:

$$\alpha_{i\to j} = \min[1, \exp(-\beta\Delta U)]$$

where $\Delta U = U_j - U_i$.

3. Why This Satisfies Boltzmann Sampling

Let's test what happens to the ratio of transition probabilities for two states where state j has a higher energy than state i ($U_j > U_i$):

1. **Going uphill ($i \to j$):** The move is accepted with probability $\alpha_{i\to j} = \exp\left[-\beta\left(U_j - U_i\right)\right]$.

2. **Going downhill ($j \to i$):** The move is always accepted, so $\alpha_{j\to i} = 1$.

If we divide the transition probabilities (assuming symmetric proposal moves where $q_{i\to j} = q_{j\to i}$), we get:

$$\frac{P_{i\to j}}{P_{j\to i}} = \frac{\alpha_{i\to j}}{\alpha_{j\to i}} = \exp\left[-\beta\left(U_j - U_i\right)\right]$$

Because the detailed balance equation requires $\frac{\pi_j}{\pi_i} = \frac{P_{i\to j}}{P_{j\to i}}$, this leads directly to:

$$\frac{\pi_j}{\pi_i} = \exp\left[-\beta\left(U_j - U_i\right)\right]$$

This ratio matches the **Boltzmann probability ratio** exactly.

4. Key Takeaways

- **Sufficient Condition:** Detailed balance is a sufficient condition to ensure that the Boltzmann distribution acts as a stable stationary distribution for the Markov chain.

- **Balanced Weights:** Lower-energy moves are naturally favored, while uphill moves are accepted with the exact mathematical probability required to preserve proper Boltzmann weights.

- **The Bottom Line:** Metropolis samples correctly because its transition rules satisfy detailed balance with respect to the Boltzmann distribution.

class:

Detailed balance:

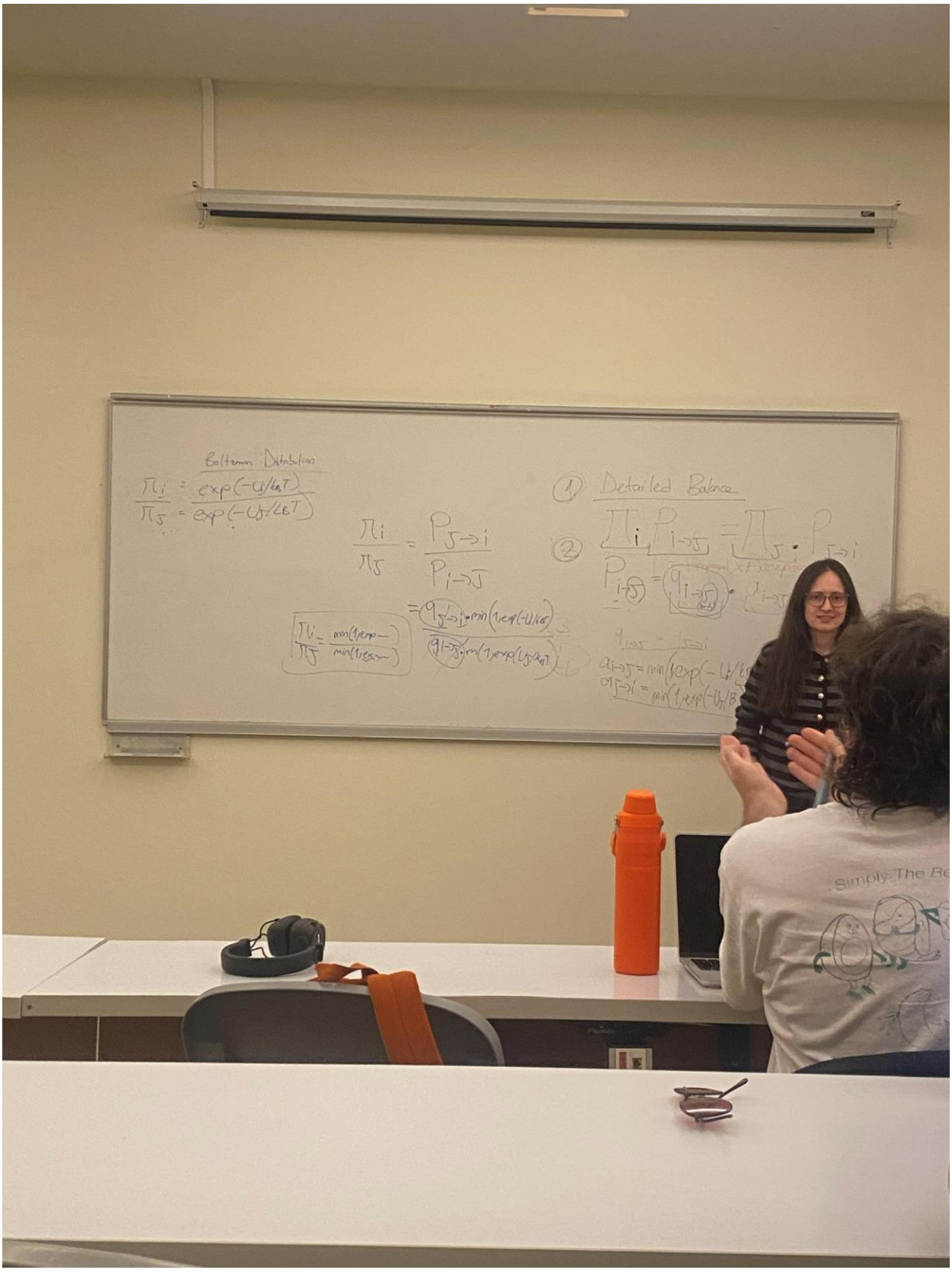
The probability of transition from i to j is lik the inverse one in equilibrium. The equations are important in concepts and for knowing them. q and a represent what, etc.

Proposed movement from I to j and the acceptance rate.

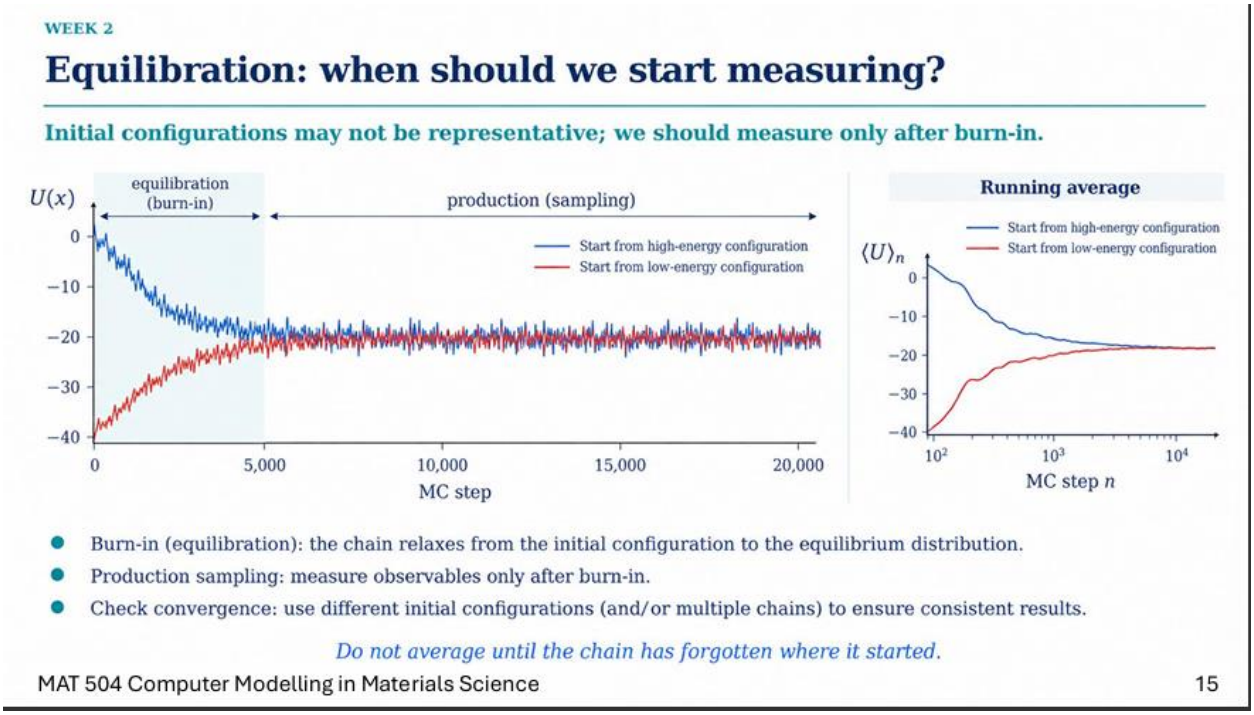
The proposed move attempt is the same not the probability of going from I to j; j to I has a different meaning.

We should consider the min, not just the exp.

Metropolis satisfies the detailed balance.


Boltzman Distribution
Detailed Balance
Simply The Be

Slide 15:



Explanation:

the core concept is **equilibration** (also known as "burn-in") in computational simulations, such as Monte Carlo (MC) methods. The fundamental rule is that early simulation data is biased by its arbitrary starting position and must be discarded; you must wait until the system "forgets" where it started before calculating averages.

The Two Simulation Phases

- **Equilibration (Burn-in):** The initial period (shown as steps 0 to ~5,000 in the left graph) where the simulation chain "relaxes" from its artificial starting configuration toward a natural equilibrium distribution.

- **Production (Sampling):** The stable phase that follows burn-in (steps 5,000 to 20,000+). During this phase, the system naturally fluctuates around a stable mean, and this is the only time you should measure observables.

Convergence Verification

To prove a simulation has actually reached equilibrium, the graphs illustrate a standard verification technique:

- **Divergent Starting Points:** The left graph tracks potential energy $U(x)$ for two separate simulation chains: one starting from a high-energy configuration (blue line) and one from a low-energy configuration (red line).

- **Running Averages:** The right graph tracks the running average ($\langle U \rangle_n$) of these two chains over the MC steps on a logarithmic scale. Over time, both lines converge to the exact same horizontal asymptote, proving the system has reached a stable equilibrium regardless of its starting point.

Usage Guidelines for Simulations

When setting up simulations (as taught in MAT 504 Computer Modelling in Materials Science), apply these steps to ensure accurate results:

1. **Initialize multiple chains** from drastically different starting configurations to ensure robust testing.

2. **Monitor an observable metric**, such as energy, as the steps progress.

3. **Identify the convergence point** where the different chains meet and stabilize.

4. **Discard the initial data** (the burn-in phase) up to that convergence point.

5. **Average your data** only from the subsequent production sampling phase.

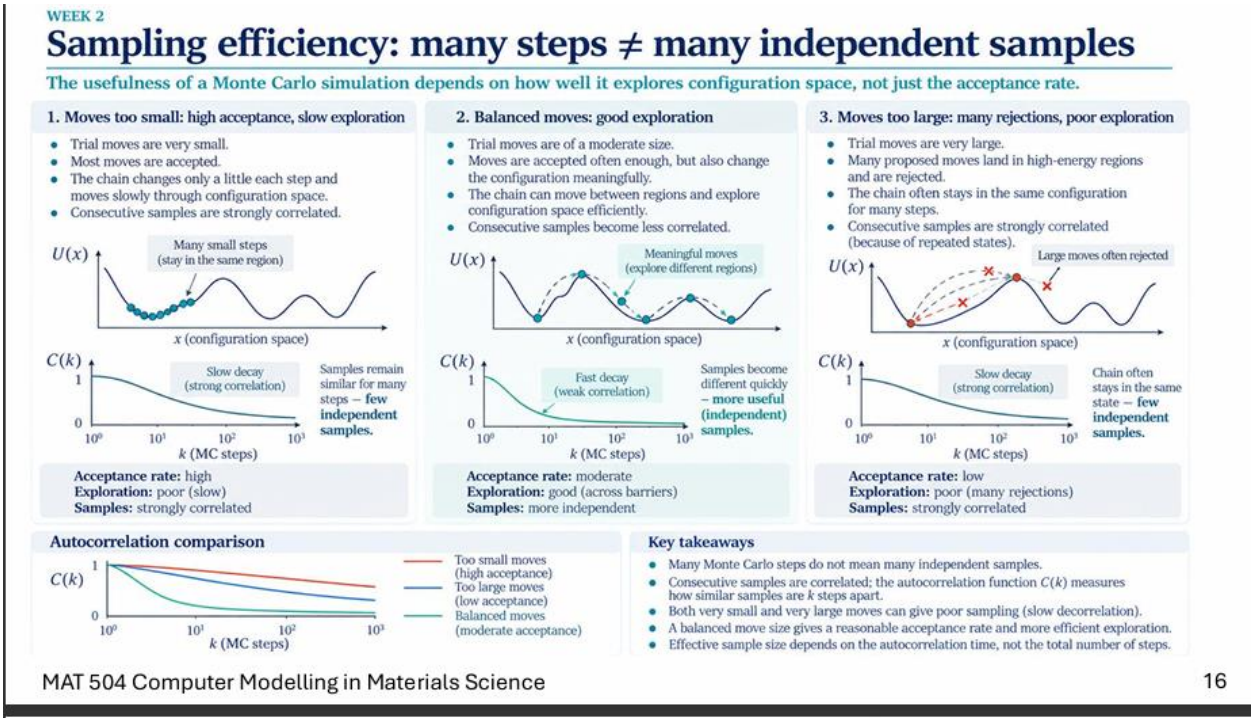
Class:

The system needs to reach its minimum energy; then we can start sampling and computing.

When two plots reach each other means the equilibrium state of the system.

Many more, not just these two initial configurations.

Slide 16:



The concept of trial move size (Δx) in Monte Carlo simulations perfectly mirrors the **learning rate** in machine learning. The efficiency of a simulation depends on how effectively it explores the "configuration space" (the energy landscape, similar to an ML loss function), rather than

simply having a high acceptance rate. The slide emphasizes that taking many steps does not automatically generate many independent samples.

Here is how the step size (Δx) dictates sampling efficiency:

- **Moves too small (Like a tiny learning rate):** Trial moves are very small, meaning most are accepted. However, the system moves too slowly and gets trapped exploring a single local minimum. Because it stays in the same region, consecutive samples are strongly correlated, resulting in slow and inefficient exploration.

- **Moves too large (Like an oversized learning rate):** Trial moves are massive, causing the system to frequently land in invalid, high-energy regions. These moves are heavily rejected, causing the chain to stay stuck in the exact same state for many consecutive steps. Just like taking steps that are too small, this results in poor exploration and strongly correlated samples.

- **Balanced moves (The optimal learning rate):** Trial moves are a moderate size. They are accepted often enough to progress, but are large enough to change the configuration meaningfully and hop between different local minima. This allows the chain to move between different regions efficiently, producing valuable, independent samples that are less correlated.

Class:

Delta x is matter mean our value of steps like learning rate in ml :

Very little change: not efficient; trapped in one local minimum.

A larger one same problem because of larger steps.

Here is a synthesized final overview combining the technical definitions with your notes on history dependence.

The Role of Autocorrelation in Monte Carlo Simulations

Autocorrelation measures how long it takes for the "history dependence" of a system to disappear. In a simulation, if you only change one small thing (like a single atom's position), the system's overall properties remain nearly identical to the previous step. This creates a high correlation, meaning the next step is heavily dependent on the current one.

Evaluating Sampling Efficiency

To maximize efficiency, the goal is to achieve a **lower autocorrelation** (a fast decay to near zero), which generates genuinely independent data rather than redundant, correlated states.

- **Small Moves:** Create a slow decay because the system barely changes from step to step, keeping historical dependency high.

- **Large Moves:** Also create a slow decay because many moves are rejected, forcing the system to freeze in the exact same state for multiple steps.

- **Balanced Moves:** Produce a fast decay, breaking the history dependence quickly and efficiently dropping the correlation to zero.

Ultimately, your simulation's **effective sample size** is dictated entirely by how fast this autocorrelation decays, not by the sheer volume of steps you simulate. A lower autocorrelation time directly yields a higher effective sample size, marking a successful and efficient simulation.

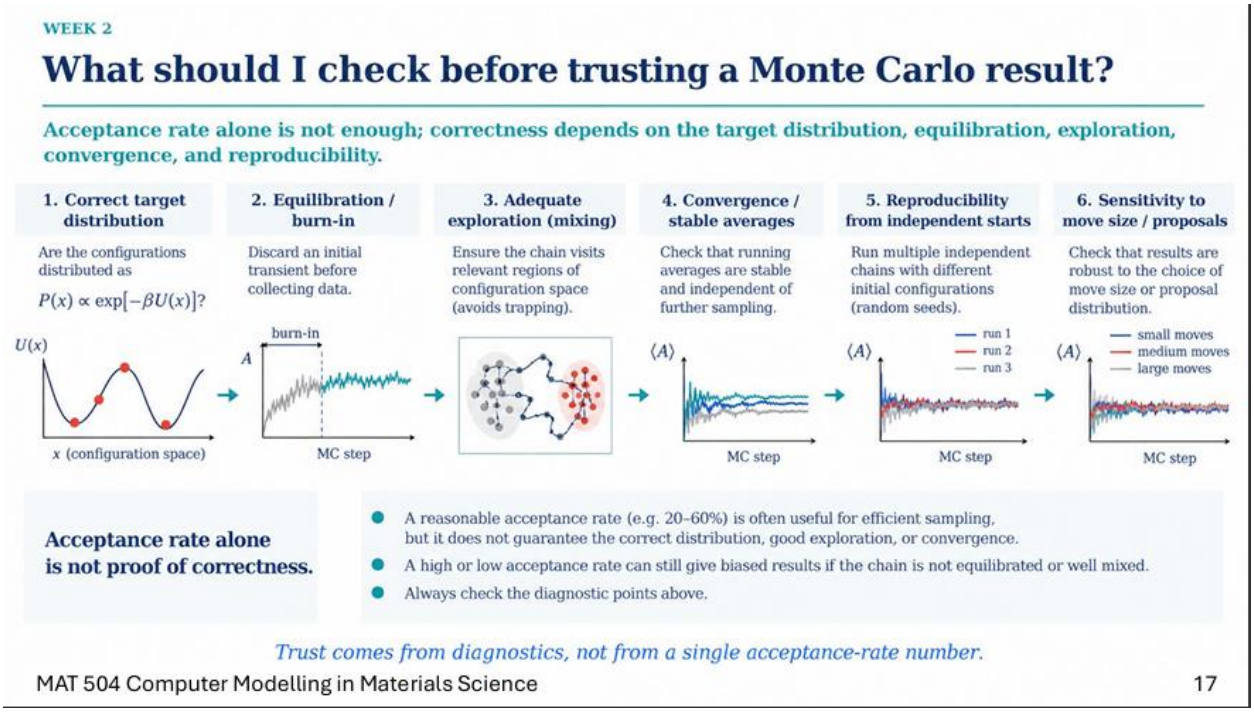
Autocorrelation:

Allows us to measure how long of a time that history dependence to disappear.

Correlation between steps: a high one means more dependency to next step.

Small changes are very correlated with the next and current steps. Say one atom changes position, so the current and next system have similar properties (large correlation)

Slide17:



Monte Carlo simulation requires evaluating six distinct diagnostic checks, as relying on the acceptance rate alone is insufficient to prove correctness.

Diagnostic Checks for Monte Carlo Results

- **1. Correct target distribution:** Verify that the configurations are distributed according to the expected probability, $P(x) \propto \exp[-\beta U(x)]$.

- **2. Equilibration / burn-in:** Discard the initial transient data (the burn-in phase) before you begin collecting measurements.

- **3. Adequate exploration (mixing):** Ensure the simulation chain successfully visits all relevant regions of the configuration space to avoid getting trapped in local minima.

- **4. Convergence / stable averages:** Confirm that the running averages ($\langle A \rangle$) stabilize and become independent of further sampling steps. :

  Independent of further steps: Once a system has truly converged, running the simulation for thousands of additional steps will not meaningfully change the final average. If the value is still trending upward or downward as you add more steps, it has not converged, meaning the simulation needs to run longer before the data can be trusted.

- **5. Reproducibility from independent starts:** Run multiple independent simulation chains starting from different initial configurations (using random seeds) to ensure they all converge to the identical result.

- **6. Sensitivity to move size / proposals:** Check that your final results are robust and consistent regardless of the chosen move size (small, medium, or large).

The Limits of Acceptance Rates

The core takeaway is that **acceptance rate alone is not proof of correctness**. While maintaining a reasonable acceptance rate (e.g., 20-60%) is useful for making the sampling efficient, it does not guarantee proper convergence, correct distribution, or adequate exploration. Simulations with high or low acceptance rates can still produce highly biased results if the chain is poorly mixed or not fully equilibrated. Trusting the result requires confirming all six diagnostic points.

In the context of these Monte Carlo simulations, the **acceptance rate** is the proportion of proposed trial moves that the algorithm accepts rather than rejects.

Here is how it functions based on the provided slides:

- **Relationship to Move Size:** Small trial moves result in a high acceptance rate because most proposals are small enough to be valid, whereas very large trial moves result in a low acceptance rate because they frequently land in high-energy regions and get rejected.

- **Ideal Range:** A moderate or "reasonable" acceptance rate is typically between 20% and 60%, which is generally useful for achieving efficient sampling.

- **Not Proof of Correctness:** The acceptance rate alone cannot guarantee a correct simulation. A simulation with a seemingly good (or high/low) acceptance rate can still produce biased results if the system is not fully equilibrated or fails to explore the configuration space adequately.

Evaluation or sampling: is more about our expectation

1- Meet the different expected conformations for a specific system, say a protein

Last one: different ones reach convergence at the same point.

Based on the whiteboard notes in equation.jpg, here is the explanation of detailed balance and how Monte Carlo satisfies it, using the exact parameters written by the instructor to prevent confusion.

The Detailed Balance Equation

Detailed balance ensures that the flow of probability between two states is equal at equilibrium. On the board, this fundamental rule is labeled as equation (1):

$$\pi_i \cdot P_{i \to j} = \pi_j \cdot P_{j \to i}$$

Here are the specific variables used on the board:

- π_i **and** π_j: The target equilibrium probabilities for states i and j.

- $P_{i \to j}$: The total transition probability to move from state i to state j.

Breaking Down the Transition Probability

To run a Monte Carlo simulation, the total transition probability $P_{i \to j}$ is split into two parts: a proposal and an acceptance.

$$P_{i \to j} = q_{i \to j} \cdot a_{i \to j}$$

- $q_{i \to j}$: The probability of proposing a move from i to j.

- $a_{i \to j}$: The probability of accepting that proposed move.

Solving the Equation

To prove how the Monte Carlo method satisfies detailed balance, the board outlines the following algebraic steps:

1. Rearrange Detailed Balance: Divide both sides of the detailed balance equation to isolate the target probabilities:

$$\frac{\pi_i}{\pi_j} = \frac{P_{j\to i}}{P_{i\to j}}$$

2. Substitute the Transition Breakdown: Replace P with the $q \cdot a$ breakdown:

$$\frac{\pi_i}{\pi_j} = \frac{q_{j\to i} \cdot a_{j\to i}}{q_{i\to j} \cdot a_{i\to j}}$$

3. Apply Symmetric Proposals: The board assumes the proposal distribution is symmetric, meaning proposing a move forward is just as likely as proposing a move backward ($q_{i\to j} = q_{j\to i}$). Because they are equal, the q terms cancel each other out (shown explicitly crossed out on the board), leaving:

$$\frac{\pi_i}{\pi_j} = \frac{a_{j\to i}}{a_{i\to j}}$$

4. Introduce the Target Distribution (Boltzmann): The left side of the whiteboard defines the target probabilities using the Boltzmann distribution based on the system's energy (U) and thermal energy ($k_B T$):

$$\frac{\pi_i}{\pi_j} = \frac{\exp(-U_i/k_B T)}{\exp\left(-U_j/k_B T\right)}$$

5. The Metropolis Acceptance Rule: To make the ratio of acceptances ($\frac{a_{j\to i}}{a_{i\to j}}$) perfectly match the ratio of the Boltzmann probabilities ($\frac{\pi_i}{\pi_j}$), the Metropolis algorithm defines the acceptance probability $a_{i\to j}$ using a minimum function:

$$a_{i\to j} = \min(1,\text{exp ratio})$$

By using this specific $\min(1, \dots)$ rule for $a_{i\to j}$, the mathematics naturally force the transition rates to match the Boltzmann distribution, perfectly satisfying the detailed balance equation.